

Tessera

written by AI, hardened against AI

Manuale di Tessera

Scrivere Tessera a mano · Tessera 0.2

Stefano Stegani - Airgrid Network - www.airgrid.it

Apache 2.0 · DOI 10.5281/zenodo.23855820 · github.com/stegani/tessera

Questo manuale è per chi vuole scrivere Tessera **a mano**, senza un'IA. Descrive il linguaggio così come lo implementa davvero il prototipo 0.2: ogni programma completo di queste pagine viene controllato ed eseguito dai test del progetto. Le idee alla base del linguaggio, e le parti previste ma non ancora costruite, sono nel paper.

I nomi nel codice sono in inglese, come nella versione inglese del manuale: il codice è identico nelle due versioni.

Indice

1. Installazione e primo programma
2. File, moduli e nomi
3. Funzioni
4. Valori e tipi
5. Variabili
6. Controllo del flusso e logica
7. Testo
8. Liste
9. Tipi propri
10. match
11. Errori come valori: Option e Result
12. Contratti ed esempi
13. Effetti e processi
14. Proprietà: edit e take
15. Secret e Untrusted
16. Compilare in WebAssembly
17. Messaggi di errore
18. Non ancora disponibile
19. Chi arriva da un altro linguaggio

1. Installazione e primo programma

Serve Python 3.12 o successivo. Dalla cartella del progetto:

```
python -m venv .venv
```

```
.venv/Scripts/pip install -r requirements.txt
```

Su Linux e macOS il percorso è `.venv/bin/pip`. Poi attiva l'ambiente e salva questo come `hello.tsr`:

```
module hello

process main
  effects: console
do
  print("Hello, Tessera!")
end process main
```

```
python -m tessera run hello.tsr
```

Stampare è un *effetto*, quindi il programma lo deve dichiarare: `effects: console`. La sezione 13 spiega perché.

I comandi:

Comando	Cosa fa
`python -m tessera run file.tsr`	controlla il programma, poi esegue il suo `process` o la sua `fn main()`
`python -m tessera check file.tsr`	solo i controlli statici, senza eseguire (`--json` per un errore JSON per riga)
`python -m tessera test file.tsr`	esegue ogni clausola `example:` come test
`python -m tessera build file.tsr -o build`	compila in WebAssembly (`.wat` e `.wasm`)
`python -m tessera call file.tsr nome arg...`	chiama una funzione con gli argomenti dati (`--wasm` per la versione compilata)

I comandi hanno anche nomi italiani: `esegui`, `verifica`, `prova`, `compila`, `chiama`.

2. File, moduli e nomi

- Un file è un modulo e comincia con `module nome` (sono ammessi nomi con il punto: `module shop.prices`).
- Un'istruzione per riga, niente `;`. Dentro le parentesi si può andare a capo liberamente. L'indentazione non conta per il parser, ma per convenzione è di due spazi.
- I commenti cominciano con `--` e arrivano a fine riga.
- **I nomi dei tipi sono in `PascalCase`, tutti gli altri in `snake_case`.** Lo impone la grammatica: `parse_price`, `ParseError`, `max_items`.
- Il codice è solo ASCII; le stringhe possono contenere qualsiasi carattere.
- `pub` rende una funzione, un tipo o una costante visibile agli altri moduli. Per usarla altrove si importa per nome (mai `*`):

```
from shop.prices import Cents, parse_price, total
```

I moduli importati si cercano tra i file `.tsr` della stessa cartella e delle sottocartelle, in base al loro nome `module`. examples/checkout.tsr importa da examples/prices.tsr.

Le costanti si dichiarano a livello di modulo, con il loro tipo:

```
const max_items: i64 = 100
```

3. Funzioni

```
module functions

pub fn area(width: i64, height: i64) -> i64
  doc: "Area of a rectangle."
  effects: none
  requires: width >= 0 and height >= 0
  ensures: result >= 0
  example: area(3, 4) == 12
do
  return width * height
end fn area
```

La firma è tutto ciò che serve a chi chiama la funzione, sempre nello stesso posto e in un **ordine fisso**:

Clausola	Obbligatoria	Significato
`doc: "..."`	no	documentazione
`effects: none` oppure `effects: console, fs`	**sì**	cosa può fare la funzione oltre a calcolare (sezione 13)
`requires: condizione`	no, ripetibile	cosa deve garantire chi chiama; controllata all'ingresso
`ensures: condizione`	no, ripetibile	cosa garantisce la funzione; controllata all'uscita, `result` è il valore restituito
`example: espressione`	no, ripetibile	un test che deve essere vero; si esegue con `tessera test`

Poi `do`, il corpo, e `end fn` seguito dal nome della funzione. Altre regole:

- `return` è sempre esplicito, e ogni percorso che restituisce un valore deve arrivare a un `return`.
- Una funzione senza `-> Tipo` non restituisce nulla.
- I parametri sono in sola lettura; la sezione 14 mostra come modificare la variabile di chi chiama.

- Non ci sono classi né metodi da dichiarare: `x.f(a)` è un altro modo di scrivere `f(x, a)`.
- Le funzioni anonime si scrivono `fn(x) => x * 2` e si usano con `map` e `filter`.

4. Valori e tipi

Tipo	Valori	Note
`i64`	`42`, `-7`, `1_000_000`	interi a 64 bit; l'overflow **ferma il programma**, non ricomincia mai dal fondo
`i32`, `i16`, `i8`, `u64`, `u32`, `u16`, `u8`		controllati dove il tipo è dichiarato (parametri, valori restituiti, variabili con tipo)
`f64`	`2.5`, `1.0`	va scritto il punto decimale: `2.0`, non `2`
`bool`	`true`, `false`	
`Str`	`"testo"`	sezione 7
`List[T]`	`[1, 2, 3]`	sezione 8
`(A, B)`	`(1, "a")`	tuple
`Option[T]`	`Some(x)`, `None`	un valore o niente (sezione 11)
`Result[T, E]`	`Ok(x)`, `Err(e)`	un valore o un errore (sezione 11)

Regole dell'aritmetica:

- **Interi e decimali non si mescolano mai**: `1 + 2.0` è un errore. Si converte con `n.to_f64()` o `x.to_int()`.
- `/` tra interi tronca verso zero e `%` ha il segno del dividendo: `-7 / 2` vale `-3`, `-7 % 2` vale `-1`.
- La divisione per zero ferma il programma, anche tra decimali.
- Le quantità hanno l'unità: `512MB`, `4GB`, `30s`, `200ms` (`1KB` vale 1024 byte).

Metodi dei numeri: `to_f64()`, `to_str()`, `abs()` sugli interi; `to_int()`, `to_str()`, `abs()`, `floor()`, `round()`, `sqrt()` sui decimali. Funzioni di base: `min(a, b)`, `max(a, b)`, `abs(x)`, `from_code(n)` (il carattere con codice `n`).

5. Variabili

```
module variables

fn sum_to(n: i64) -> i64
  effects: none
  example: sum_to(4) == 10
do
  let start = 1
  var total = 0
  for i in start..n + 1 do
    total += i
  end for
  return total
end fn sum_to

fn evens(limit: i64) -> List[i64]
  effects: none
  example: evens(7) == [0, 2, 4, 6]
do
  var out: List[i64] = []
  for i in 0..limit do
    if i % 2 == 0 do
      out.push(i)
    end if
  end for
  return out
end fn evens
```

- `let` dichiara un valore che non cambia mai; `var` uno che può cambiare.
- Assegnazione: `=`, `+=`, `-=`, `*=`, `/=`. Non c'è `++`.
- Una lista vuota ha bisogno del tipo: `var out: List[i64] = []`.
- **Valori, non riferimenti**: `let b = a` crea una copia indipendente. Se poi cambia `a`, `b` resta com'era.
- `let (x, y) = pair` scompone una tupla.

6. Controllo del flusso e logica

`else` si può scrivere anche `else do`, come specchio di `if ... do`: sono accettati entrambi.

```
module control

fn classify(n: i64) -> Str
  effects: none
  example: classify(-3) == "negative"
  example: classify(0) == "zero"
  example: classify(12) == "large"
do
  if n < 0 do
    return "negative"
  else if n == 0 do
    return "zero"
  else if n > 10 do
    return "large"
  else
    return "small"
  end if
end fn classify

fn first_multiple(values: List[i64], k: i64) -> i64
  effects: none
  requires: k != 0
  example: first_multiple([3, 5, 10, 12], 5) == 5
  example: first_multiple([1, 2], 7) == -1
do
  var found = -1
  for v in values do
    if v % k != 0 do
      continue
    end if
    found = v
    break
  end for
  return found
end fn first_multiple

fn countdown(n: i64) -> List[i64]
  effects: none
  example: countdown(3) == [3, 2, 1]
do
  var out: List[i64] = []
  var i = n
  while i > 0 do
    out.push(i)
    i -= 1
```

```
  end while
  return out
end fn countdown

fn in_range(x: i64, low: i64, high: i64) -> bool
  effects: none
  example: in_range(5, 1, 10)
  example: not in_range(10, 1, 10)
do
  return low <= x and x < high
end fn in_range
```

- Ogni blocco finisce con `end` più la sua parola chiave: `end if`, `end for`, `end while`, `end match`.
- `for i in 0..n` conta da 0 a n-1: **gli intervalli escludono la fine**. Da 1 a n si scrive `1..n + 1`.
- `for x in lista` scorre una lista; `for c in text.chars()` i caratteri di un testo.
- La logica si scrive a parole: `and`, `or`, `not`, `implies`.
- `and` e `or` **non si mescolano senza parentesi**: si scrive `(a and b) or c`.
- **I confronti non si concatenano**: si scrive `low <= x and x < high`, mai `low <= x < high`.
- `x in [1, 2, 3]` controlla l'appartenenza.

7. Testo

```
module text

fn greet(name: Str, count: i64) -> Str
  effects: none
  example: greet("Ada", 2) == "Hello Ada, you have 2 messages"
do
  return "Hello {name}, you have {count} messages"
end fn greet

fn initials(full_name: Str) -> Str
  effects: none
  example: initials("ada lovelace") == "AL"
do
  var out = ""
  for word in full_name.split(" ") do
    if not word.is_empty() do
      out = "{out}{word.slice(0, 1).upper()}"
    end if
  end for
  return out
end fn initials
```

- Le stringhe usano le virgolette doppie. **L'interpolazione è sempre attiva**: `"{name}"`, `"{a + b}"`. Per scrivere una graffa la si protegge: `\{` e `\}`.
- Altre sequenze: `\"`, `\\`, `\n`, `\t`.
- Non c'è `+` tra stringhe: si uniscono con l'interpolazione (`"{a}{b}"`) o con `parts.join("")`.
- Un carattere è uno `Str` di una lettera: `"c"`, mai `'c'`.

Metodo	Risultato
`len()`, `is_empty()`	lunghezza, vuoto o no
`upper()`, `lower()`, `trim()`	copia modificata
`contains(s)`, `starts_with(s)`, `ends_with(s)`	`bool`
`replace(a, b)`	copia con ogni `a` sostituito
`split(sep)`	`List[Str]`
`split_once(sep)`	`(Str, Str)`: prima e dopo il primo `sep` (dopo vale `""` se manca)
`chars()`	`List[Str]`, uno per carattere
`slice(start, end)`	una parte del testo; fuori dai limiti ferma il programma
`code()`	il codice di un testo di un carattere; `from_code(n)` fa il contrario
`is_digits()`	non vuoto e solo cifre ASCII 0-9
`to_int()`	`Result[i64, Str]`
`to_float()`	`Result[f64, Str]`

8. Liste

```
module lists

fn positives(values: List[i64]) -> List[i64]
  effects: none
  example: positives([3, -1, 0, 7]) == [3, 7]
do
  return values.filter(fn(v) => v > 0)
end fn positives

fn squares(values: List[i64]) -> List[i64]
  effects: none
  example: squares([1, 2, 3]) == [1, 4, 9]
do
  return values.map(fn(v) => v * v)
end fn squares

fn total(values: List[i64]) -> i64
  effects: none
  example: total([]) == 0
  example: total([4, 5]) == 9
do
  if values.is_empty() do
    return 0
  end if
  return values.sum()
end fn total
```

Metodo	Risultato
`len()`, `is_empty()`	lunghezza, vuota o no
`get(i)`	`Option`: `Some(x)`, oppure `None` fuori dalla lista
`first()`, `last()`	`Option`
`list[i]`	l'elemento; **fuori dalla lista il programma si ferma**
`contains(x)`	`bool`
`map(f)`, `filter(f)`	nuova lista
`sum()`	la somma; si ferma su una lista vuota, quindi prima si controlla `is_empty()`
`sorted()`, `reversed()`, `slice(start, end)`	nuova lista
`join(sep)`	unisce una `List[Str]`
`push(x)`, `pop()`, `clear()`	modificano la lista: solo su una `var` o su un parametro `edit`; `pop()` restituisce un `Option`

9. Tipi propri

```
module shapes

pub type Point do
  x: i64
  y: i64
end type Point

pub type Shape do
  case Circle(radius: i64)
  case Rectangle(width: i64, height: i64)
  case Empty
end type Shape

pub type Percent
  invariant: value >= 0 and value <= 100
do
  value: i64
end type Percent

pub type Cents = i64

fn manhattan(a: Point, b: Point) -> i64
  effects: none
  example: manhattan(Point(x: 0, y: 0), Point(x: 3, y: -4)) == 7
do
  return abs(a.x - b.x) + abs(a.y - b.y)
end fn manhattan

fn area(s: Shape) -> i64
  effects: none
  example: area(Shape.Rectangle(width: 2, height: 3)) == 6
  example: area(Shape.Empty) == 0
do
  match s do
    case Circle(r):
      return 3 * r * r
    case Rectangle(w, h):
      return w * h
    case Empty:
      return 0
  end match
end fn area
```

```
fn apply(price: Cents, discount: Percent) -> Cents
  effects: none
  example: apply(Cents(1000), Percent(value: 10)) == Cents(900)
do
  return price - price * discount.value / 100
end fn apply

fn euros(amount: Cents) -> i64
  effects: none
  example: euros(Cents(1250)) == 12
do
  let Cents(c) = amount
  return c / 100
end fn euros
```

- Le **strutture** elencano i loro campi; si costruiscono per nome, `Point(x: 1, y: 2)`, o per posizione, `Point(1, 2)`; un campo si legge con `p.x`.
- I **tipi somma** elencano i loro casi con `case`, con o senza campi; si costruiscono con `Shape.Empty` o `Shape.Rectangle(width: 2, height: 3)`.
- Gli **invarianti** vanno prima di `do` e si controllano ogni volta che si costruisce un valore: `Percent(value: 120)` ferma il programma.
- **Tipi distinti**: `type Cents = i64` *non* è un sinonimo. Un `Cents` e un `i64` non si mescolano per sbaglio: `Cents(5)` ne costruisce uno, `let Cents(c) = amount` ne tira fuori il numero. Tra due `Cents` l'aritmetica funziona come al solito, e un `Cents` si può moltiplicare o dividere per un numero semplice.

10. match

```
module matching

type Command do
  case Move(steps: i64)
  case Turn(degrees: i64)
  case Stop
end type Command

fn describe(c: Command) -> Str
  effects: none
  example: describe(Command.Move(steps: 0)) == "stay"
  example: describe(Command.Move(steps: 3)) == "move 3"
  example: describe(Command.Stop) == "stop"
do
  match c do
    case Move(n) if n == 0:
      return "stay"
    case Move(n):
      return "move {n}"
    case Turn(d):
      return "turn {d}"
    case Stop:
      return "stop"
  end match
end fn describe

fn sign_name(n: i64) -> Str
  effects: none
  example: sign_name(0) == "zero"
  example: sign_name(5) == "other"
do
  match n do
    case 0:
      return "zero"
    case _:
      return "other"
  end match
end fn sign_name
```

- `case Schema:` e poi le istruzioni sulle righe seguenti. Gli schemi possibili: una variante con i suoi campi (`Move(n)`), un letterale (`0` , `"yes"`), una tupla (`(a, b)`), un nome che prende qualsiasi valore, oppure `_` .

- Una guardia aggiunge una condizione: `case Move(n) if n == 0:`.
- **Un match deve coprire tutti i casi**: dimenticare una variante è un errore che nomina quella mancante. Si aggiunge il caso o un `case _:` finale.
- Per controllare una sola forma senza un match completo: `value is Some(x)` dà un `bool`.

11. Errori come valori: Option e Result

Tessera non ha eccezioni né `null`. Una funzione che può fallire lo dice nel tipo che restituisce.

```
module parsing

pub type AgeError do
  case Empty
  case NotANumber
  case TooOld(age: i64)
end type AgeError

fn parse_age(text: Str) -> Result[i64, AgeError]
  effects: none
  example: parse_age("42") == Ok(42)
  example: parse_age("") == Err(AgeError.Empty)
  example: parse_age("4x") == Err(AgeError.NotANumber)
  example: parse_age("200") == Err(AgeError.TooOld(age: 200))
do
  if text.is_empty() do
    return Err(AgeError.Empty)
  end if
  if not text.is_digits() do
    return Err(AgeError.NotANumber)
  end if
  let age = text.to_int().unwrap_or(0)
  if age > 150 do
    return Err(AgeError.TooOld(age: age))
  end if
  return Ok(age)
end fn parse_age

fn age_gap(a: Str, b: Str) -> Result[i64, AgeError]
  effects: none
  example: age_gap("30", "42") == Ok(12)
  example: age_gap("30", "") == Err(AgeError.Empty)
do
  let x = parse_age(a)?
  let y = parse_age(b)?
  return Ok(abs(x - y))
end fn age_gap

fn first_even(values: List[i64]) -> Option[i64]
  effects: none
  example: first_even([1, 3, 4, 6]) == Some(4)
  example: first_even([1, 3]) == None
do
  for v in values do
    if v % 2 == 0 do
```

```
      return Some(v)
    end if
  end for
  return None
end fn first_even
```

- `r?` dà il valore dentro `Ok` o `Some`, oppure restituisce subito `Err` o `None`. Funziona solo dentro una funzione che restituisce lo stesso genere (`Result` o `Option`).
- `unwrap_or(predefinito)` dà il valore, oppure `predefinito` in caso di `Err` o `None`.
- `is_ok()`, `is_err()`, `is_some()`, `is_none()` controllano il genere.
- **Un `Result` non si può ignorare**: un'istruzione che produce un `Result` senza usarlo è un errore. Si gestisce con `?`, `match` o `unwrap_or`.

12. Contratti ed esempi

I contratti trasformano le ipotesi in controlli:

- `requires:` si controlla quando la funzione viene chiamata: se è falsa, l'errore è di *chi chiama*.
- `ensures:` si controlla quando la funzione restituisce; `result` è il valore restituito. Con `implies` una condizione vale solo in certi casi: `ensures: result is Ok(c) implies c >= 0` lega `c` per la parte a destra.
- `invariant:` (sezione 9) si controlla quando si costruisce un valore del tipo.
- Le clausole `example:` sono test che vivono nella firma: `python -m tessera test file.tsr` le esegue tutte.

Un contratto violato ferma il programma con un messaggio che nomina la funzione e la condizione. examples/prices.tsr li mostra tutti insieme.

13. Effetti e processi

Ogni funzione che fa qualcosa oltre a calcolare lo deve dichiarare in `effects:`. Il prototipo conosce due effetti:

Effetto	Funzione di base
`console`	`print(valore)`
`fs`	`read_text(percorso)`, che restituisce `Result[Str, Str]`

Una funzione con `effects: none` non può chiamare `print`, né qualsiasi funzione che dichiari un effetto: l'errore dice quale effetto aggiungere, e dove. Così si sa dalla sola firma che una funzione non tocca né i file né lo schermo.

Il programma parte da un `process` (o da una `fn main()`). Un processo dice anche quali **permessi** riceve:

```
process main
  effects: console, fs
  grants: fs.read("data")
do
  match read_text("data/welcome.txt") do
    case Ok(text):
      print("Read: {text.trim()}")
    case Err(problem):
      print("Error: {problem}")
  end match
end process main
```

- `grants: fs.read("data")` permette di leggere solo sotto la cartella `data`; qualsiasi altro percorso viene rifiutato. Fuori da un processo non c'è alcun permesso sui file.
- I percorsi relativi partono dalla cartella del file principale, non da quella da cui si lancia il comando.
- `platform: ecc, spectre_mitigations` elenca cosa deve garantire la macchina: la *sentinella hardware* lo controlla prima dell'avvio, e si rifiuta di partire se un requisito non si può verificare.
- `limits: memory 512MB` viene letto ma non è ancora applicato.

14. Proprietà: edit e take

I parametri sono in sola lettura. Due parole, ripetute alla chiamata perché si veda cosa cambia:

```
module ownership

fn add_tax(edit prices: List[i64], percent: i64)
  effects: none
do
  var i = 0
  while i < prices.len() do
    prices[i] = prices[i] + prices[i] * percent / 100
    i += 1
  end while
end fn add_tax

fn close(take items: List[Str]) -> Str
  effects: none
  example: close(["a", "b"]) == "a, b"
do
  return items.join(", ")
end fn close

fn main()
  effects: console
do
  var prices = [100, 250]
  add_tax(edit prices, 10)
  print(prices)
  var names = ["bolts", "screws"]
  print(close(take names))
end fn main
```

- `edit` lavora sulla variabile di chi chiama, che deve essere una `var`.
- `take` la consuma: dopo `close(take names)`, usare di nuovo `names` è un errore. Un valore temporaneo (una lista scritta sul momento, il risultato di una chiamata) si può passare a un parametro `take` senza scrivere `take`.
- La stessa variabile non può comparire due volte in una chiamata se una delle due è `edit` o `take`.
- `take` di una variabile esterna dentro un ciclo è un errore: al giro successivo il valore non ci sarebbe più.
- Non ci sono durate (*lifetime*) né riferimenti dentro le strutture: il modello è tutto qui.

15. Secret e Untrusted

```
module security

fn check_pin(pin: Secret[Str], attempt: Str) -> bool
  effects: none
  example: check_pin(Secret("1234"), "1234")
  example: not check_pin(Secret("1234"), "0000")
do
  return pin == Secret(attempt)
end fn check_pin

fn main()
  effects: console
do
  let pin = Secret("1234")
  print(check_pin(pin, "0000"))
  let input = Untrusted("42")
  match input.validate(fn(s) => s.is_digits()) do
    case Ok(text):
      print("valid: {text}")
    case Err(problem):
      print("rejected: {problem}")
  end match
end fn main
```

- Un `Secret` si può passare e confrontare con un altro `Secret`, ma il suo contenuto non esce mai: **stamparlo o interpolarlo è un errore**, così una password non può finire in un log per sbaglio.
- `Untrusted` marca i dati che vengono da fuori (un utente, la rete, l'uscita di un modello linguistico). L'unico modo di usarne il contenuto è `validate(controllo)`, che restituisce `Ok` solo se il controllo passa.

16. Compilare in WebAssembly

```
python -m tessera build examples/numbers.tsr -o build
```

```
python -m tessera call examples/numbers.tsr gcd 48 18 --wasm
```

Il compilatore tratta le funzioni con `effects: none` che usano solo `i32`, `i64`, `f64` e `bool`: variabili, `if`, `while`, `for` sugli intervalli, chiamate tra funzioni compilate, `to_f64`, `to_int`, `abs` e contratti. Le altre funzioni vengono saltate, e il compilatore dice perché. Il codice compilato mantiene le stesse

garanzie: overflow, un contratto violato o una divisione per zero lo fermano con una trappola, esattamente come si ferma l'interprete. I test confrontano i due su argomenti casuali.

17. Messaggi di errore

Ogni errore ha un genere, una riga e, dove possibile, un suggerimento:

Genere	Esempi
sintassi	un blocco chiuso fuori ordine, `let mut`, `&&`, una clausola al posto sbagliato
tipo	interi e decimali mescolati, un `Result` ignorato, un `match` che manca un caso
effetti	`print` in una funzione con `effects: none`
proprietà	modificare un `let`, usare un valore dopo `take`
contratto	un `requires`, un `ensures` o un `invariant` falso
esecuzione	overflow, divisione per zero, indice fuori dai limiti
sicurezza	stampare un `Secret`, `unsafe` fuori da un modulo kernel

Con `--json` gli errori arrivano uno per riga, in una forma leggibile dai programmi (e dai modelli). Gli errori di sintassi sono in inglese, con un suggerimento preciso per le abitudini prese da altri linguaggi. **Nel prototipo 0.2 gli altri messaggi sono ancora in italiano**; verranno tradotti.

18. Non ancora disponibile

Il paper descrive più di quanto implementi il prototipo. Non disponibili nella 0.2:

- il controllo statico dei tipi (i tipi si controllano durante l'esecuzione, ai confini delle funzioni);
- interfacce per le funzioni generiche, mappe, rete, `llm`, tensori e derivazione;
- dimostrazioni formali, funzioni esterne (`extern`), pacchetti;
- l'applicazione di `limits` e l'identità IPv6 dei processi (lette, non applicate).

I blocchi `unsafe` esistono solo nei moduli con `profile: kernel`.

19. Chi arriva da un altro linguaggio

Sbagliato	Giusto
`fn f(x: i64) -> i64 {`	firma, poi `effects: none` su una riga propria, poi `do`
`let mut x = 0`	`var x = 0`
`if x > 0 then` / `if x > 0:`	`if x > 0 do` … `end if`
`elif`	`else if x > 0 do`
`end` da solo	`end if`, `end for`, `end while`, `end match`, `end fn nome`
`&&`, `||`, `!x`	`and`, `or`, `not x`
`0 <= x < n`	`0 <= x and x < n`
`1..=n`	`1..n + 1`
`|x| x * 2`, `lambda x: x * 2`	`fn(x) => x * 2`
`x++`, `;`	`x += 1`, niente
`"a" + "b"`	`"{a}{b}"`
`'c'`	`"c"`
`str`, `int`, `Vec<i64>`, `String`	`Str`, `i64`, `List[i64]`
`case Ok(n) => ...`	`case Ok(n):` e le istruzioni sulle righe seguenti
`len(x)`, `x.len`	`x.len()`
`null`, eccezioni	`Option`, `Result`
`# commento`, `// commento`	`-- commento`

Tessera è creato da **Stefano Stegani - Airgrid Network - www.airgrid.it**. Licenza Apache 2.0, con l'avviso di attribuzione in NOTICE. DOI 10.5281/zenodo.23855820.